

Relatório de Modernização Arquitetural

Projeto: Clube do Pão (MVP)

Disciplina: Engenharia de Software - AI Assisted Development

Professor: Luiz Real

Equipe:

- Danilo Sebastiany França
- Marcos Vinicius Souza Lima
- Kevin Peterson Coelho
- Victor Bastos dos Santos

1. Visão Executiva

Este relatório documenta o processo de refatoração e modernização arquitetural do MVP "Clube do Pão" (Release 2). Utilizando princípios de Engenharia de Software Moderna (Clean Code, SOLID, DRY, YAGNI) e o suporte de agentes de Inteligência Artificial (Antigravity), o projeto evoluiu de um protótipo funcional para uma base de código escalável, testável e de alta coesão.

A metodologia aplicada garantiu que nenhuma refatoração fosse realizada sem uma rede de segurança estrita. Todas as iterações foram validadas contra um arquivo de registro de integridade (`baseline.md`), garantindo a preservação total do comportamento visual e funcional da aplicação, validada por testes automatizados e builds de produção.

2. Diagnóstico Técnico Guiado por IA

A primeira etapa consistiu em uma análise estática autônoma, onde o agente de IA atuou sob diretrizes restritas (sem alterar código) para mapear débitos técnicos e *code smells*.

Foram identificados 9 apontamentos, dos quais 3 foram classificados com **Severidade Alta** e priorizados para o ciclo de modernização:

1. **Frontend (App.jsx): Componente único e lógica concentrada.** A aplicação inteira residia em um único arquivo (God Component), acoplando roteamento, layout e lógica de negócio.
2. **Frontend (App.jsx): Violação da separação de responsabilidades.** Lógica de chamadas HTTP e regras de negócio embutidas diretamente na camada de renderização visual (UI).
3. **Backend (database.js): Estado global compartilhado.** Conexão SQLite mantida em variável global, gerando forte acoplamento e risco de falhas de concorrência (*Database is locked*).

3. Execução da Refatoração (Frontend)

O frontend em React foi o primeiro alvo da modernização, visando a aplicação do princípio de Responsabilidade Única (SRP - SOLID) e a separação clara entre a Camada de Apresentação e a Camada de Controle.

3.1. Quebra do Monólito (Componentização)

- **Ação:** O arquivo `App.jsx` foi desmembrado.
- **Resultado:** O `App.jsx` assumiu exclusivamente a responsabilidade de roteamento base. As interfaces foram isoladas em componentes dedicados: `PageShell.jsx`, `HomePage.jsx`, `CustomerView.jsx` e `BakeryDashboard.jsx`.
- **Métrica de Sucesso:** O build do Vite foi concluído com sucesso e a cobertura de testes de linha subiu de **40.36% para 42.19%**, evidenciando que componentes menores são mais facilmente interpretados pelo motor de testes.

3.2. Abstração de Lógica de Domínio (Custom Hooks)

- **Ação:** Remoção da lógica de domínio e estados globais de dentro dos componentes de tela recém-criados.
- **Resultado:** Criação da estrutura `/hooks`. A lógica do formulário de clientes foi extraída para `useCustomer.js` e a lógica do dashboard da padaria para `usePcp.js`.
- **Impacto Arquitetural:** Os componentes visuais tornaram-se "burros" (*dumb components*), focados apenas na renderização do Tailwind CSS, consumindo os estados e métodos gerenciados exclusivamente pelos *Custom Hooks*. A cobertura se manteve estável em **42.12%**.

4. Execução da Refatoração (Backend)

No Node.js, o foco foi mitigar o risco arquitetural mais grave: a gestão de estado do banco de dados, preparando a aplicação para lidar com concorrência segura.

4.1. Injeção de Dependência e Isolamento de Estado

- **Ação:** Refatoração do módulo `database.js` e seus controladores dependentes.
- **Resultado:** A conexão SQLite deixou de ser uma variável global mutável. O módulo foi reescrito para fornecer a conexão através de Injeção de Dependência para os controladores que necessitam interagir com a base de dados.
- **Métrica de Sucesso:** O *cold start* da aplicação foi validado (servidor subiu na porta 3000 sem travamentos de *lock* do SQLite).

4.2. Estabelecimento de Baseline do Backend

Durante a validação da refatoração, identificou-se que o backend carecia de um registro formal de métricas. A suíte de testes foi executada utilizando o test runner nativo do Node.js (`node --test`), estabelecendo o seguinte *baseline* oficial:

- **Testes:** 3/3 passando (0 falhas)
- **Cobertura de Linhas:** 70.11%
- **Cobertura de Branches:** 65.31%
- **Cobertura de Funções:** 77.42%

5. Conclusão da Entrega

A aplicação das técnicas de *AI Assisted Development* demonstrou que o uso de Large Language Models (LLMs) em engenharia de software alcança sua eficiência máxima quando orquestrado por diretrizes humanas rigorosas. A imposição de *prompts* restritivos (exigindo validação de cobertura de testes, builds funcionais e leitura cruzada de documentações) impediu alucinações e degradação de código.

O "Clube do Pão" encerra a Release 2 não apenas com novas funcionalidades logísticas validadas, mas com uma arquitetura resiliente, de alta manutenibilidade e tecnicamente madura.